

Week 2 Quiz

AI SYSTEMS DESIGN & INFERENCE ENGINEERING · MEWTWO COHORT

Class 2 Practice Quiz. Anatomy of an LLM Call

15 questions · ~20 minutes · answers with explanations at the end. No grade. this is a self-check before the lab class and homework.

Part A. Concepts (multiple choice)

Q1. Your inference stack has three layers: client, gateway, engine. Which layer owns the **KV cache**?

- a) The model. The KV values are part of the trained weights
- b) The engine. It creates and manages KV state per request at runtime
- c) The gateway. It caches keys and values for repeated prompts
- d) The client. It carries conversation state between turns

Q2. A request arrives with an API key the system has never seen. In a correctly layered system, **where** is it rejected?

- a) The engine refuses to schedule it
- b) The gateway rejects it before it ever reaches the engine
- c) The client library blocks it locally
- d) It runs, but usage is billed to an anonymous account

Q3. Why is messages an array instead of a single prompt string?

- a) So the server can parallelize prefill across messages
- b) Because the client carries conversation history. the server is stateless and each request re-sends the whole conversation
- c) Because the KV cache requires one entry per message
- d) To allow multiple users to share one request

Q4. In a streaming response, TTFT is measured at the moment...

- a) ...the data: [DONE] sentinel arrives
- b) ...the HTTP connection is established
- c) ...the first delta chunk arrives
- d) ...prefill finishes on the server

Q5. A streaming connection dies at chunk 40 of 100. In class we said this is the hard case for gateways. Why?

- a) The engine must re-run prefill, which is expensive
- b) With an engine hooked directly to the app, nobody records that the request failed mid-stream.retry/flag decisions need a layer that keeps that record
- c) SSE cannot resume, so the response is lost forever
- d) The client's 40 chunks must be un-rendered

Q6. Per the rule of thumb discussed in class (and confirmed from Bloomberg/Cisco experience), where do REST and gRPC each belong?

- a) gRPC at the edge, REST between microservices
- b) REST at the edge for users; gRPC inside, between your own services
- c) REST everywhere.gRPC is deprecated
- d) gRPC everywhere once you have more than one service

Q7. A RAG application ingests long documents and returns short summaries. Which phase dominates, and what's the observable symptom?

- a) Decode-bound.inter-token latency explodes
- b) Prefill-bound.TTFT is large, but tokens flow smoothly once generation starts
- c) Neither.RAG is network-bound
- d) Both equally, always

Q8. "Agentic workloads are CPU-bound." What does the CPU actually do in an agent system?

- a) It runs the forward pass when the GPU is full
- b) Tool calling, routing, planning, orchestration.everything except the model's forward pass

- c) Only tokenization
- d) Nothing.the statement is false

Part B.The codelab (multiple choice)

Q9. In the warm-up you ran curl <http://127.0.0.1:8081/v1/chat/completions> and later chatted in Open WebUI at :3000. What's the relationship between these two actions?

- a) The browser talks to llama.cpp; curl talks to the naive server
- b) They're both clients speaking the same OpenAI-shaped contract.the UI is just another client, like curl
- c) Open WebUI bypasses HTTP and calls the engine directly
- d) curl is for health checks only; real requests need the UI

Q10. Why does the codelab run llama.cpp from a prebuilt Docker image instead of building it with CMake?

- a) llama.cpp cannot be built on Mac at all
- b) Docker makes inference faster
- c) CMake/Xcode builds are temperamental, and the rule is: never build an engine from source unless you're contributing to it
- d) The license requires containerization

Q11. The naive server (naive_server/server.py) has a MODEL_LOCK that serializes the GPU. Why does it still OOM under concurrency?

- a) The lock leaks memory on every acquire
- b) The lock serializes COMPUTE, but the server still accepts unlimited requests.each accepted request allocates and holds its own KV cache while waiting
- c) Transformers ignores locks entirely
- d) It doesn't OOM.it only gets slow

Q12. In the break-the-server lab, why must the T3 metrics poller (part5_observability.py) start **before** the T2 concurrency sweep?

- a) The poller warms up the model, reducing cold start

b) Otherwise active= reads zero the whole time and you miss the memory climb.the heart-rate monitor goes on before the stress test

c) Modal requires metrics before accepting load

d) The sweep script crashes if /metrics was never called

Q13. System D is LiteLLM.a genuinely good production gateway.pointed at the naive Transformers backend. Under the same sweep that killed System A, what happens, and why does the lab include it?

a) It survives.LiteLLM queues requests, which prevents OOM

b) It fails like A.D is the control experiment proving gateway features are not engine fixes

c) It survives but with higher TTFT

d) It fails differently.LiteLLM's retries cause a retry storm

Part C.System Design

Q14. Between System A (naive) and System C (RelayServe → llama.cpp), two things changed. Name both, and say which observed improvement each one is responsible for.

Q15. You're on call for an inference service. GPU utilization has been pinned at 100% for an hour. Your teammate wants to page the team. Using what you learned in class, give (a) the reason 100% GPU util alone is not an incident, and (b) the two metrics you'd check instead to decide.

Answer key

Part A.Concepts

Q1.b The model is just weights; the KV cache is runtime state the engine creates and manages per request. (Cleared up live in class.it is not “embedded in the model”, and nothing needs activating.)

Q2.b Auth is policy, and policy lives at the gateway. The engine attends to every request it receives, in order, with no concept of clients.someone must say no before it, and that someone is the gateway.

Q3.b State belongs to the client. Every turn re-sends the full conversation, which is why context and cost grow per turn and why chat “memory” is a client-maintained illusion. (Our naive server just grabs the last user message.)

Q4.c Time to first token = first delta's arrival. (a) is total time; (d) happens server-side and is invisible to the client.TTFT as the client sees it also includes queueing and network.

Q5.b The engine just stops; without a middle layer, no component records the partial failure or decides retry-vs-flag. This ambiguity is exactly why streaming makes gateways hard and few do it well.

Q6.b REST at the edge (ubiquitous, zero client codegen), gRPC inside (strongly typed Protobuf, better latency/throughput under load, bidirectional streaming). Fair correction from class: gRPC IS language-agnostic via codegen. The real difference is that REST needs no generated clients at all.

Q7.b Long docs in = heavy prefill; short summary out = light decode. Symptom: big TTFT, fine inter-token latency. Caveat from class: it depends on output length. Ask for an essay and decode grows. Classify by where the tokens are, not the product name.

Q8.b Execution. Tool calls, routing, planning, workflow orchestration. Runs on CPU; only the forward pass (tokenization onward) is GPU work. This is also why hardware vendors are pushing disaggregated prefill and faster CPU↔GPU links (PCIe → NVLink → C2C; Vera Rubin).

Part B. The codelab

Q9.b Same backend, same contract, two clients. Port 8081 = llama.cpp (warm-up engine), 8000 = our naive FastAPI server, 3000 = Open WebUI, which is a stock Docker image wired to the naive server by ~15 lines of docker-compose (OPENAI_API_BASE_URL=<http://naive-server-cpu:8000/v1>). There is no chat-UI code in the repo.

Q10.c Prebuilt image from the GitHub container registry, run in Docker. Reproducible for everyone in class. Build from source only to contribute. (The venv, by contrast, holds YOUR Python side: FastAPI, clients, scripts. Two layers, two packaging worlds.)

Q11.b The lock protects the GPU's compute and does nothing for its memory. Unlimited admission × per-request KV cache = memory climbs with every accepted request until OOM. The other two flaws: one generate() per request (no batching), and API + engine in one process (no queue limit, no backpressure, no admission control).

Q12.b Observability must precede the failure, or you learn nothing from it. This is also the production lesson in miniature: System A's death was a mystery polled from outside; System C would have shown queue depth climbing before users felt pain.

Q13.b Same OOM as A. One variable at a time: B isolates the engine (survives ⇒ engine credit), D isolates the gateway (dies ⇒ gateway alone is not the fix). Attribution rules: engine owns KV, batching, tok/s, OOM resistance; gateway owns contract, routing, queue, metrics.

Part C. System Design

Q14. (1) The engine changed: Transformers → llama.cpp.responsible for surviving the sweep: OOM resistance (quantized GGUF weights leave headroom, slot-based continuous batching, tight preallocated KV management) and tokens/sec. (2) A gateway appeared: RelayServe.responsible for the queue (requests wait instead of stacking in memory), /metrics (queue depth, backend stats), the relay metadata per response (backend, queue_ms, ttft_ms), and one stable contract. Full credit requires attributing each improvement to the correct layer. "RelayServe fixed the OOM" is exactly the error System D exists to disprove.

Q15. (a) 100% utilization is a COST metric, not a health metric. a fully busy GPU is efficient if latency holds; and note the naive server died at LOW utilization (memory-bound + serialized), so util is misleading in both directions. (b) Check TTFT p95 against the SLA (are users feeling pain?) and queue depth (the leading indicator. it grows before latency explodes). Also acceptable: OOM/503 rate, tokens/sec, KV/GPU memory. Page on symptoms users feel and predictors that lead them. not on the GPU being busy.

If you missed Q11, Q13, or Q14. reread notes Section 11. Those three are the heart of the class: unlimited admission × per-request KV = death; and credit the engine for physics, the gateway for operations. Reflection Q2 of the homework is graded against exactly this.